Solution: Staircase

Problem Statement

Given a stair with 'n' steps, implement a method to count how many possible ways are there to reach the top of the staircase, given that, at every step you can either take 1 step, 2 steps, or 3 steps.

Example 1:

```
Number of stairs (n) : 3
Number of ways = 4
Explanation: Following are the four ways we can climb : {1,1,1}, {1,2},
{2,1}, {3}
```

Example 2:

```
Number of stairs (n) : 4
Number of ways = 7
Explanation: Following are the seven ways we can climb : {1,1,1,1},
{1,1,2}, {1,2,1}, {2,1,1},
{2,2}, {1,3}, {3,1}
```

Let's first start with a recursive brute-force solution.

Basic Solution

At every step, we have three options: either jump 1 step, 2 steps, or 3 steps. So our algorithm will look like this:

Java

```java
class Solution {

  public int countWays(int n) {
    if( n == 0)
      return 1; // base case, we don't need to take any step, so there is only one way

    if(n == 1)
      return 1; // we can take one step to reach the end, and that is the only way

    if(n == 2)
      return 2; // we can take one step twice or jump two steps to reach at the top

    // if we take 1 step, we are left with 'n-1' steps;
    int take1Step = countWays(n-1);
    // similarly, if we took 2 steps, we are left with 'n-2' steps;
    int take2Step = countWays(n-2);
    // if we took 3 steps, we are left with 'n-3' steps;
    int take3Step = countWays(n-3);

    return take1Step + take2Step + take3Step;
  }
```

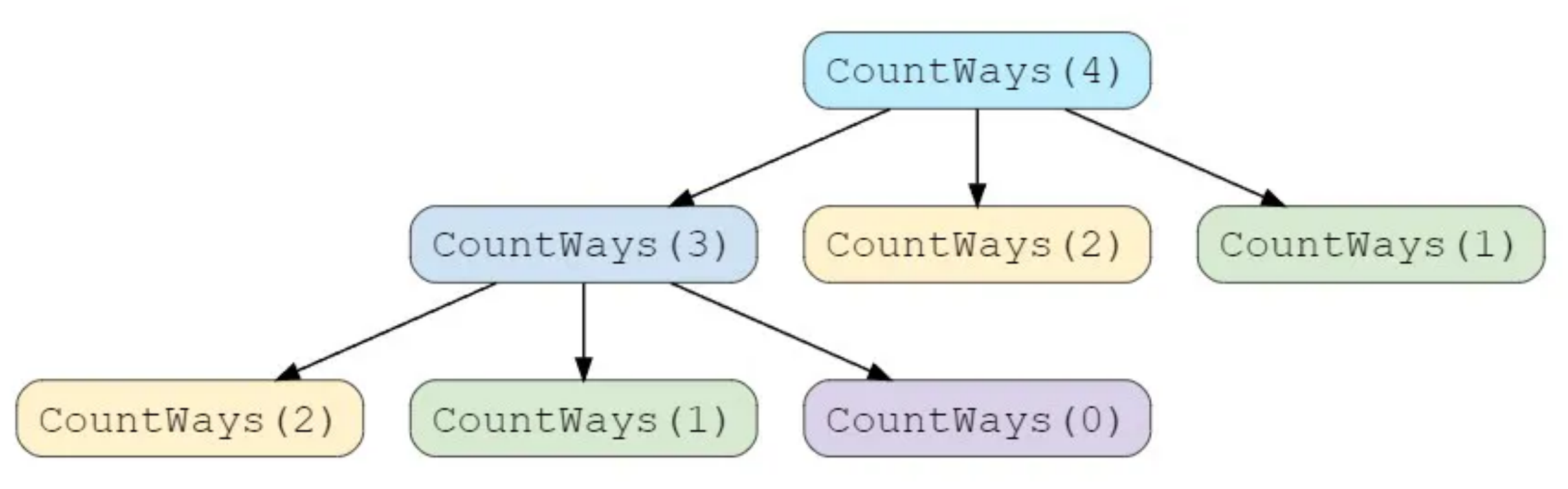
Run Code Result Stdout

You must run your code first

Execute

The time complexity of the above algorithm is exponential $O(3^n)$ as we are making three recursive call in the same function. The space complexity is $O(n)$ which is used to store the recursion stack.

Let's visually draw the recursion for `CountWays(4)` to see the overlapping subproblems:



Recursion tree for calculating ways to climb stairs

We can clearly see the overlapping subproblem pattern: `CountWays(2)` and `CountWays(1)` have been called twice. We can optimize this using memoization.

Top-down Dynamic Programming with Memoization

We can use an array to store the already solved subproblems. Here is the code:

Java

```java
class Solution {

  public int countWays(int n) {
    int dp[] = new int[n+1];
    return countWaysRecursive(dp, n);
  }

  public int countWaysRecursive(int[] dp, int n) {
    if( n == 0)
      return 1; // base case, we don't need to take any step, so there is only one way

    if(n == 1)
      return 1; // we can take one step to reach the end, and that is the only way

    if(n == 2)
      return 2; // we can take one step twice or jump two steps to reach at the top

    if(dp[n] == 0) {
      // if we take 1 step, we are left with 'n-1' steps;
      int take1Step = countWaysRecursive(dp, n-1);
      // similarly, if we took 2 steps, we are left with 'n-2' steps;
```

Run Code Result Stdout

You must run your code first

Execute

What is the time and space complexity of the above solution? Since our memoization array `dp[n+1]` stores the results for all the subproblems, we can conclude that we will not have more than $n + 1$ subproblems (where 'n' represents the total number of steps). This means that our time complexity will be $O(N)$. The space complexity will also be $O(n)$; this space will be used to store the recursion-stack.

Bottom-up Dynamic Programming

Let's try to populate our `dp[]` array from the above solution, working in a bottom-up fashion. As we saw in the above code, every `CountWaysRecursive(n)` is the sum of the previous three counts. We can use this fact to populate our array.

Code

Here is the code for our bottom-up dynamic programming approach:

Java

```java
class Solution {

  public int countWays(int n) {
    if (n < 2) return 1;
    if (n == 2) return 2;

    int dp[] = new int[n+1];
    dp[0] = 1;
    dp[1] = 1;
    dp[2] = 2;

    for(int i=3; i<=n; i++)
      dp[i] = dp[i-1] + dp[i-2] + dp[i-3];

    return dp[n];
  }

  public static void main(String[] args) {
    Solution sc = new Solution();
    System.out.println(sc.countWays(3));
    System.out.println(sc.countWays(4));
```

Run Code Result Stdout

You must run your code first

Execute

The above solution has time and space complexity of $O(n)$.

Memory optimization

We can optimize the space used in our previous solution. We don't need to store all the counts up to 'n', as we only need three previous numbers to calculate the next count. We can use this fact to further improve our solution:

Java

```java
class Solution {

  public int countWays(int n) {
    if (n < 2) return 1;
    if (n == 2) return 2;

    int n1=1, n2=1, n3=2, temp;
    for(int i=3; i<=n; i++) {
      temp = n1 + n2 + n3;
      n1 = n2;
      n2 = n3;
      n3 = temp;
    }
    return n3;
  }

  public static void main(String[] args) {
    Solution sc = new Solution();
    System.out.println(sc.countWays(3));
    System.out.println(sc.countWays(4));
    System.out.println(sc.countWays(5));
```

Run Code Result Stdout

You must run your code first

Execute

The above solution has a time complexity of $O(n)$ and a constant space complexity $O(1)$.

Fibonacci number pattern

We can clearly see that this problem follows the Fibonacci number pattern. The only difference is that in Fibonacci numbers every number is a sum of the two preceding numbers, whereas in this problem every count is a sum of three preceding counts. Here is the recursive formula for this problem:

```
CountWays(n) = CountWays(n-1) + CountWays(n-2) + CountWays(n-3), for n ⩾3
```

This problem can be extended further. Instead of taking 1, 2, or 3 steps at any time, what if we can take up to 'k' steps at any time? In that case, our recursive formula will look like:

```
CountWays(n) = CountWays(n-1) + CountWays(n-2) + CountWays(n-3) + ... + CountWays(n-k),
for n ⩾ k
```

← Previous
Staircase

Next →
Number Factors

☐ Mark as Completed